

안녕하세요!

Unity용 종합 치트 방지 솔루션인 **Anti-Cheat Toolkit (ACTk)** 을 구입해 주셔서 감사합니다!

부인 성명:

이 플러그인에 사용되는 치트 방지 기술은 100% 안전하고 깨지지 않는 척하지 않습니다(클라이언트 측에서는 불가능합니다). 하지만 대부분의 치트 사용자가 앱을 해킹하려는 시도를 막아야 합니다. 명심하세요: 의욕이 넘치고 숙련된 해커는 무엇이든 깨뜨릴 수 있습니다!

스크립팅 측면에서 플러그인 API 사용에 대해 질문이 있는 경우: codestage.net/uas_files/actk/api



v. 2023.2.6 [버

[전 내역\]](#)

내용물

- [설치 및 설정](#)
- [메모리 치트 방지](#)
- [저장 치트 방지](#)
- [플레이어 기본 설정 치트 방지](#)
- [플레이어 기본 설정 및 가려진 기본 설정 편집기 창](#)
- [공통 장치 잠금 기능 참고 사항](#)
- [코드 난독화 및 전반적인 코드 보호 참고사항](#)
- [안드로이드 앱 설치 소스 검증](#)
- [코드 무결성 검증](#)
- [일반적인 치트 탐지기 기능 및 설정](#)
- [가려진 유형의 부정 행위 감지](#)
- [스피드 해킹 감지](#)
- [시간 부정 행위 감지](#)
- [월해 탐지](#)
- [관리형 DLL 주입 감지](#)
- [타사 플러그인 통한 및 메모](#)
- [문제 해결](#)
- [마이그레이션 노트](#)
- [호환성](#)
- [저자의 마지막 말](#)
- [치트 방지 툴킷 링크 및 지원](#)

설치 및 설정

새 버전을 프로젝트로 가져오기 전에 이전 버전을 완전히 제거하여 새 버전의 파일 구조 변경으로 인한 문제를 방지하세요.

플러그인을 프로젝트로 가져온 후 추가 설정 및 제어를 위한 새 메뉴를 찾을 수 있습니다.

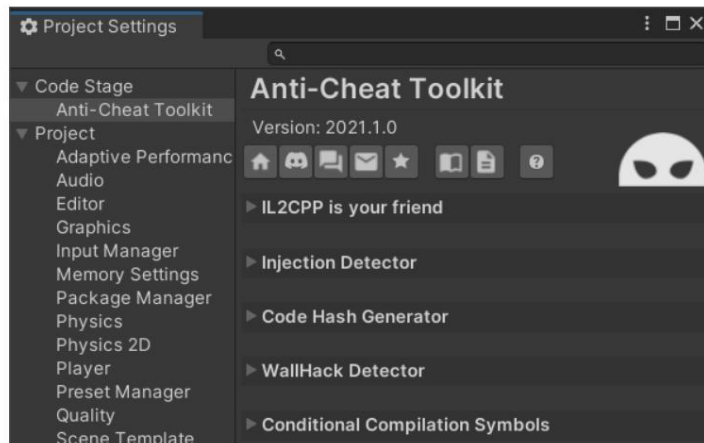
- [도구 > 코드 스테이지 > 치트 방지 툴킷](#)
- [GameObject > 기타 생성 > 코드 스테이지 > Anti-Cheat 툴킷](#)

플러그인 기능, 모범 사례 및 힌트에 대해 자세히 알아보려면 이 문서를 읽어보세요.

설정 창

편집기 기본 설정 섹션을 사용하여 디버깅 및 호환성 목적으로 감지기와 조건부 컴파일 기호를 구성합니다. 모든 설정은 프로젝트의 "[ProjectSettings/ACTkSettings.asset](#)"에 저장됩니다.

ACTk 설정을 열려면 [도구 > 코드 스테이지 > 치트 방지 툴킷 > 설정](#) 메뉴 명령을 사용하면 됩니다.



여기에서 다음과 같은 항목을 볼 수 있습니다.

- 프로젝트에서 사용 중인 ACTk 버전
- 유용한 링크 및 바로가기(홈페이지, Discord, 포럼, 지원, 리뷰, 본 매뉴얼, API 참조, 정보)
- IL2CPP는 친구 섹션입니다. 치트 방지 관점에서 Mono에 비해 IL2CPP의 장점을 설명하고 쉽게 현재 플랫폼이 지원하는 경우 IL2CPP로 전환합니다.
- 주입 감지기 설정 섹션 - [관리되는 DLL 주입 감지](#) 장에서 자세한 내용을 참조하세요.
- 코드 해시 생성기 - [코드 무결성 검사](#) 장에서 자세한 내용을 확인하세요.
- WallHack 감지기 설정 섹션 - 자세한 내용은 [Wallhack 감지](#) 장을 참조하세요.
- 다양한 디버그 및 호환성 조정, 로그 및 변경 사항을 전환할 수 있는 조건부 컴파일 기호 섹션 그런.

플러그인 기능 심층

메모리 치트 방지 [비디오 튜토리얼]

중요: 일반 유형을 모호한 유형으로 교체할 때는 주의하세요. 검사기 값이 재설정됩니다!

이는 다양한 플랫폼에서 가장 널리 사용되는 부정 행위 방법입니다. 사람들은 메모리에서 변수를 검색하고 그 값을 변경하기 위해 특별한 도구를 사용합니다. 일반적으로 돈, 건강, 점수 등입니다. 몇 가지 예: Cheat Engine, ArtMoney(PC), Game CIH, Game Guardian(Android). 이러한 플랫폼과 다른 플랫폼을 위한 다른 도구도 많이 있습니다.

메모리 치트 방지 기술을 활용하려면 일반 유형 대신 [ObscuredInt](#), [ObscuredFloat](#), [ObscuredString](#) 등 **모호한** 유형을 사용하세요 (모든 기본 유형 + 소수의 Unity 특정 유형이 포함됨)...

이러한 유형은 다음과 같이 일반 내장 유형 대신 사용할 수 있습니다.

```
// 이 줄을 .cs 파일 시작 부분에 바로 배치하세요! CodeStage.AntiCheat.ObscuredTypes 사용 ;
```

```
int totalCoins = 500;  
ObscuredInt receivedCoins = 100; int coinLeft  
= totalCoins - CollectedCoins;
```

```
// 다음을 인쇄합니다: "수집한 동전: 100, 왼쪽: 400"  
Debug.Log(" 수집된 코인: " + receivedCoins + ", 왼쪽: " + coinLeft);
```

모든 [int](#) <=> [ObscuredInt](#) 캐스트는 암시적으로 수행됩니다. 다른 모호한 유형에도 동일합니다!

하지만 일반 [int](#) 와 [ObscuredInt](#) 사이에는 한 가지 큰 차이점이 있습니다. 사기꾼은 ObscuredInt로 메모리에 저장된 값을 쉽게 찾아 변경할 수 없습니다. 일반 [int](#) 에 대해서는 말할 수 없는 일입니다!

사실, 부정행위자가 원하는 것을 찾아 부정행위 시도를 저지하도록 허용할 수도 있습니다. 이러한 경우 실제 가려진 값은 여전히 안전합니다. 플러그인을 사용하면 사기꾼이 원하는 경우 암호화되지 않은 진짜 변수를 찾아서 변경할 수 있습니다.

이러한 부정 행위 시도를 감지하려면 아래에 설명된 [ObscuredCheatingDetector](#)를 사용하세요.

모호한 유형 JSON 직렬화

다양한 방법을 사용하여 모호한 유형을 직렬화하는 것이 가능합니다.

뉴턴소프트 Json.NET

[Newtonsoft.Json.NET 패키지를 사용하는 경우 또는 순수](#) Json.NET 라이브러리를 사용하는 경우 번들 [ObscuredTypesNewtonsoftConverter](#) 클래스를 사용하여 Obscured Types의 암호 해독된 값 직렬화를 쉽게 추가할 수 있습니다. 다른 [JsonConverter](#)처럼 사용할 수 있습니다. _____

```
Newtonsoft.Json 사용 ;  
CodeStage.AntiCheat.ObscuredTypes.Converters 사용 ;
```

```
// 전역적으로 한 번 기본 변환기에 추가합니다. // -----  
JsonConvert.DefaultSettings = ()=> 새로운 JsonSerializerSettings {
```

```
    변환기 = { 새로운 ObscuredTypesNewtonsoftConverter() }  
};
```

```
var json = JsonConvert.SerializeObject(input); // ... var 출력 =
```

```
JsonConvert.DeserializeObject<T>(json);
```

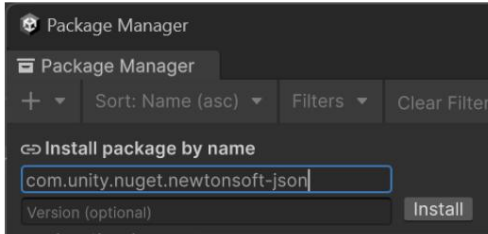
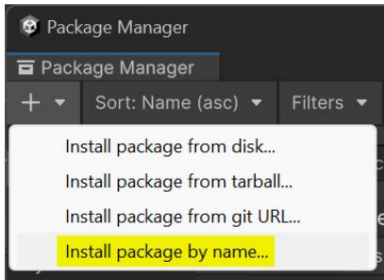
```
// 또는 재사용 가능한 JsonSerializer 변환기에 추가하세요.
// -----
var serializer = new JsonSerializer();
serializer.Converters.Add(new ObscuredTypesNewtonsoftConverter());

var json = serializer.Serialize(입력);
// ...
var 출력 = serializer.Deserialize<T>(json);

// 또는 직렬화/역직렬화할 때마다 직접 전달합니다.
// -----
var json = JsonConvert.SerializeObject(input, new ObscuredTypesNewtonsoftConverter());
// ...
var 출력 = JsonConvert.DeserializeObject<T>(json, new ObscuredTypesNewtonsoftConverter());
```

중요한:

- **ObscuredTypesNewtonsoftConverter** 는 #if ACTK_NEWTONSOFT_JSON 조건부 컴파일 로 래핑됩니다.
Unity 프로젝트에 **com.unity.nuget.newtonsoft-json** 패키지가 있는 경우에만 활성화되는 플러그 버전 2.0.0 이상이 추가되었습니다. 순수 Json.NET 라이브러리를 사용하는 경우 이 변환기를 수동으로 활성화하려면 **플레이어 설정 > 스크립팅 정의 기호** 에 **ACTK_NEWTONSOFT_JSON** 조건 기호를 추가하는 것이 좋습니다.
- 이름으로 **com.unity.nuget.newtonsoft-json** 패키지를 추가할 수 있습니다 .



- **JsonConverter** 의 대안으로 이 예에 표시된 것처럼 **ISerialized**를 구현할 수 있습니다 .

Json유틸리티

유연성은 떨어지지만 **JsonUtility**를 사용하여 모호한 유형을 직렬화하는 것은 여전히 가능합니다. **ISerializationCallbackReceiver** 사용
이 예에 표시된 것처럼 .

모호한 유형에 대한 추가 단어

모호한 유형의 사용 예를 찾을 수 있으며 "**예제/API 예**" 장면에서 직접 속이려고 시도할 수도 있습니다.
플러그인과 함께 배송됩니다.

모호한 유형에 대한 전문가 팁:

- 모든 단순 가려진 유형에는 공개 정적 메소드인 GetEncrypted() 및 SetEncrypted() 가 있어 가려진 인스턴스의 암호화된 값으로 작업할 수 있습니다. 일부 사용자 정의 저장 엔진을 사용하려는 경우 도움이 될 수 있습니다.
- 어떤 경우에는 부정 행위자가 소위 "알 수 없는 값" 검색을 사용하여 모호한 변수를 찾을 수 있습니다. 그들이 그럴 것이라는 사실에도 불구하고 암호화된 값을 찾으면 **ObscuredCheatingDetector** 가 부정 행위(치트 엔진에서 정지하거나 메모리 편집기에서 변경)를 감지하지만 여전히 찾을 수 있습니다. 이를 완전히 방지하려면 모든 기본 모호한 유형에서 RandomizeCryptoKey()라는 특수 메소드를 사용할 수 있습니다 . 변수가 변경되지 않는 순간에 원래 값을 그대로 유지하면서 암호화된 표현을 변경하는 데 사용됩니다. 사기꾼은 변경되지 않은 값을 검색하지만 실제로 값이 변경되므로 변수를 찾을 수 없습니다.
- 생성기() API를 사용하여 Encrypt(값, 키) 메서드 에 대한 임의의 암호화 키를 생성할 수 있습니다 . 잊지 마세요
해당 키를 나중에 Decrypt() 값에 저장하세요.

중요한:

- 현재 **ObscuredBigInteger**, **ObscuredBool**, **ObscureDateTime**, **ObscuredDecimal**, **ObscuredDouble** , **ObscuredFloat**, **ObscuredInt**, **ObscuredLong**, **ObscuredQuaternion**, **ObscuredShort**, **ObscuredString**, **ObscuredUInt**, **ObscuredULong**, **ObscuredVector2**, **ObscuredVector2Int**, **ObscuredVector3**, **ObscuredVector3Int**. 일반 유형을 모호한 유형으로 교체할 때는 주의하세요. 검사기 값이 재설정됩니다!

- 모호한 유형은 Unity DOTS와 호환되지 않으므로 ECS 엔터티 외부에 보관하십시오 ([예](#)).
- 여전히 매우 빠르고 가볍지만, 모호한 유형에는 일반적으로 다음 유형에 비해 추가 리소스가 필요합니다.
일반 것. 업데이트, 루프, 대규모 배열 등에서 모호한 변수를 멀리하고 특히 모바일 프로젝트 작업 시 프로파일을 주의 깊게 살펴보기 바랍니다.
- [LINQ](#) 및 [XmlSerializer](#) 는 현재 지원되지 않습니다.
- 바이너리 직렬화는 기본적으로 지원되지만 BinaryFormatter는 안전하지 않은 것으로 간주되므로 주의하세요.
- 일반적으로 고급 작업을 수행하기 위해 모호한 변수를 일반 변수로 캐스팅하고 이를 캐스팅하는 것이 좋습니다.
그 후에 다시 돌아와서 제대로 작동하는지 확인하세요.

저장 치트 방지 [\[동영상 튜토리얼\]](#)

가려진 파일

파일은 저장된 게임 및 기타 민감한 정보를 저장하는 일반적인 장소입니다.
그러나 적절한 보호가 없으면 파일은 다음과 같은 위험에 취약합니다.

- 바이너리가 아닌 파일(게임 변수, 자격 증명, 게임 내 개발 치트 등)의 민감한 데이터 식별.
- 데이터 수정(100에서 999999까지 돈을 속이는 등).
- 다른 사기꾼과 사기를 쳤거나 수정한 파일을 공유하는 행위.

[ObscuredFile](#)은 이러한 모든 위험을 다룹니다.

- 호기심이 많은 눈으로부터 데이터를 숨기기 위해 선택적으로 데이터를 암호화합니다.
 - 변조 감지 기능이 내장되어 있어 데이터 수정을 발견할 수 있습니다(암호화된 파일과 일반 파일 모두).
 - 선택적으로 데이터를 장치 ID 또는 사용자 정의 ID에 고정하여 다른 장치에서 공유된 파일을 감지합니다. [장치 잠금](#) 보기
- [자세한 내용](#) 을 알아두세요.

ObscuredFile은 원시 바이트 배열만 읽고 쓸 수 있습니다. 사용자 친화적인 PlayerPrefs(유사한 API)는 아래 [ObscuredFilePrefs](#)를 확인하세요 .

설정 및 사용법은 매우 간단합니다.

// CodeStage.AntiCheat.Storage를 [사용하여](#) .cs 파일 시작 부분에 이 줄을 배치합니다 .

```
// 기본 설정으로 새 인스턴스 생성 var secureFile = new ObscuredFile();
```

```
// 데이터 쓰기 var  
writeResult = secureFile.WriteAllBytes(abstractBytes); if (writeResult.Success)
```

```
    Debug.Log($" {secureFile.FilePath} 에 저장된 데이터 "); else
```

```
    Debug.LogError($"데이터를 저장할 수 없습니다 : {writeResult.Error}");
```

```
// 데이터 읽기 var  
readResult = secureFile.ReadAllBytes(); if (readResult.Success) //  
readResult.Data를 처리합니다. 그렇  
지 않은 경우 (readResult.CheatingDetected)
```

```
// 사기꾼 처벌, DataFromAnotherDevice \ DataIsNotGenuine readResult 속성 확인 else Debug.LogError($" 데이터를 읽는 동안 문제가 발  
생했습니다  
니다: {readResult.Error}");
```

위의 예에서 abstractBytes는 기본 설정으로 저장되고 로드되지만 [ObscuredFileSettings](#) API를 사용하여 파일 경로 및 이름, 암호화 설정, 장치 기능 설정 잠금 및 변조 방식을 자유롭게 구성할 수 있습니다.

[ObscuredFile](#)을 구성하고 사용하는 방법에 대한 자세한 내용은 [API 설명서](#)를 참조하세요 . 및 사용 예: [예제\API 예제\스크립트\런타임\UsageExamples\ObscuredFilePrefsExamples.cs](#)

중요한:

- UnityApiResultsHolder.InitForAsyncUsage(true)를 호출합니다. 백그라운드 스레드에서 가려진 파일로 작업하기 전에 메인 스레드에서.

- ObscuredFile은 추가 암호화로 인해 일반 파일 에 비해 작동 속도가 느리진다는 점에 유의하세요 . 만들다 핫 패스에서는 사용하지 마십시오.

가려진 파일 기본 설정

이는 사용하기 쉬운 일반 API를 갖춘 정적 ObscuredFile 래퍼입니다. 다양한 유형의 기본 설정을 읽고 쓸 수 있습니다(모든 기본 C# 유형, BigInteger, byte[], DateTime 및 일부 Unity 유형이 지원됨).

ObscuredFile은 사기꾼으로부터 보호되는 ObscuredFile에 기본 설정을 저장하고 저장하는 것과 동일한 기능을 모두 가지고 있습니다. 다음은 간단한 사용 예입니다.

```
// 이 줄을 .cs 파일 시작 부분에 바로 배치합니다.
CodeStage.AntiCheat.Storage를 사용합니다 .

// 기본 설정으로 초기화
ObscuredFilePrefs.Init();

// 부정행위를 들어보세요.
ObscuredFilePrefs.NotGenuineDataDetected += OnDataCheat;
ObscuredFilePrefs.DataFromAnotherDeviceDetected += OnLockCheat;

// pref 쓰기
ObscuredFilePrefs.Set("pref 키", data);

// 기본 읽기
data = ObscuredFilePrefs.Get("pref key", defaultValue);

// 또는 데이터 유형을 지정하고 defaultValue 인수를 생략합니다. // data = ObscuredFilePrefs.Get<T>("pref key");

// 필요할 때 반복할 모든 기본 설정 키를 가져옵니다.
var 키 = ObscuredFilePrefs.GetKeys();
```

ObscuredFile 예제와 유사하게 기본 설정이 사용되었으며 필요에 맞게 ObscuredFilePrefs를 구성할 수 있습니다. ObscuredFileSettings API를 사용합니다 .

중요한:

- UnityApiResultsHolder.InitForAsyncUsage(true)를 호출합니다. 백그라운드 스레드에서 가려진 파일로 작업하기 전에 메인 스레드에서.

- ObscuredFilePrefs는 추가 암호화로 인해 일반 File 또는 PlayerPrefs 에 비해 느리게 작동한다는 점을 명심하세요 . Hot Path에서는 사용을 피해주세요.

- ObscuredFilePrefs 에는 기본적으로 활성화되는 자동 저장 기능이 있습니다. 저장되지 않은 데이터의 손실 가능성을 방지하고 앱 종료(데스크톱) 및 앱 초점 상실(모바일)에 저장합니다. 모든 권한을 갖고 싶다면 ObscuredFileSettings 에서 위험을 감수하고 비활성화할 수 있습니다 .

플레이어 기본 설정 치트 방지 [\[비디오 튜토리얼\]](#)

Unity 개발자는 소량의 민감한 데이터(돈, 게임 진행 상황 등)를 저장하기 위해 PlayerPrefs(OP) 클래스를 사용하는 경우가 많지만 거의 노력하지 않고도 찾아내고 변조할 수 있습니다. 예를 들어, regedit를 열고 Windows에서 HKEY_CURRENT_USER\Software\Your Company Name\Your Game Name 으로 이동하는 것만큼 간단합니다 !

이것이 바로 내가 이 톨킷에 ObscuredPrefs(OP) 클래스를 포함하기로 결정한 이유입니다. 이를 통해 평소와 같이 데이터를 저장할 수 있지만 보기 및 변경으로부터 안전하게 보호할 수 있습니다. 이는 저장 사기꾼을 방지하기 위해 꼭 필요한 것입니다!

다음은 간단한 예입니다.

```
// 이 줄을 .cs 파일 시작 부분에 바로 배치합니다.  
CodeStage.AntiCheat.Storage를 사용합니다 .
```

```
ObscuredPrefs.Set<float>("currentLifeBarObscured", 88.4f);  
var currentLifeBar = ObscuredPrefs.Get<float>("currentLifeBarObscured");
```

```
// 다음을 인쇄합니다: "Life bar: 88.4"  
Debug.Log("수명 막대: " + currentLifeBar);
```

보시다시피, 사용 방법에는 아무런 변화가 없습니다. 모든 것이 기존의 일반 [PlayerPrefs](#) 클래스와 동일하게 작동하지만 이제 저장 내용은 사기꾼(대부분의 사기꾼)으로부터 안전합니다!
추가 기능:

- NotGenuineDataDetected 이벤트를 구독할 수 있습니다 . 저장된 데이터 무결성 위반에 대해 알 수 있습니다.
변조된 데이터를 읽는 즉시 (전체 세션에 대해) 한 번 실행됩니다.
- [OP](#)는 일반 [PP](#) 에 비해 다양한 추가 데이터 유형 (모든 기본 C# 유형, [BigInteger](#), [byte\[\]](#), [DateTime](#))을 지원합니다.
Unity 유형은 거의 지원되지 않습니다.
- 장치 잠금 기능이 지원됩니다. 자세한 내용은 [장치 잠금 메모를 참조하세요](#) .

ObscuredPrefs 전문가 팁:

- [PP](#) 에서 [OP](#) 로 쉽게 마이그레이션할 수 있습니다 . 프로젝트에서 [PP](#) 발생을 [OP](#) 로 바꾸기만 하면 됩니다. 단, ACTk 클래스에서 [PP](#)를 [OP](#) 로 바꾸지 마십시오. [OP](#)는 처음 읽을 때 [PP](#) 에 저장된 모든 데이터를 자동으로 암호화합니다 . 원래 [PP](#) 키는 기본적으로 삭제됩니다. PreservePlayerPrefs 플러그를 사용하여 이를 보존할 수 있습니다 . 이 경우 [PP](#) 데이터는 평소와 같이 [OP](#) 로 암호화되지만 원래 키는 유지되므로 [PP](#)를 사용하여 다시 읽을 수 있습니다. [API 예제](#) 장면에서 presentsPlayerPrefs가 작동하는 것을 볼 수 있습니다 .
- 일반 [PP](#) 와 [OP](#)를 혼합할 수 있습니다 (단, 다른 키 이름을 사용해야 합니다!). 중요한 데이터만 저장하려면 모호한 버전을 사용하세요. 마이그레이션하는 동안 프로젝트의 모든 [PP](#) 호출을 교체할 필요가 없으며 일반 [PP](#)는 [OP](#) 에 비해 더 빠르게 작동합니다 .
- [OP](#)를 사용하여 로컬 리더보드, 플레이어 점수, 돈 등과 같은 적절한 양의 데이터를 저장하고 지도 배열, 데이터베이스 등과 같은 대량의 데이터를 저장하는 데 [OP](#)를 사용하지 마십시오. 이를 위해 [ObscuredFile](#) 또는 [ObscuredFilePrefs](#)를 사용하십시오.
- [ArrayPrefs2](#) 와 같이 일반 [PlayerPrefs](#)를 확장하는 데 큰 기여를 했습니다 . 예를 들어. [OP](#)는 할 수 있다
이러한 클래스에서는 [PP](#)를 쉽게 교체하여 저장된 모든 데이터를 안전하게 보호할 수 있습니다.

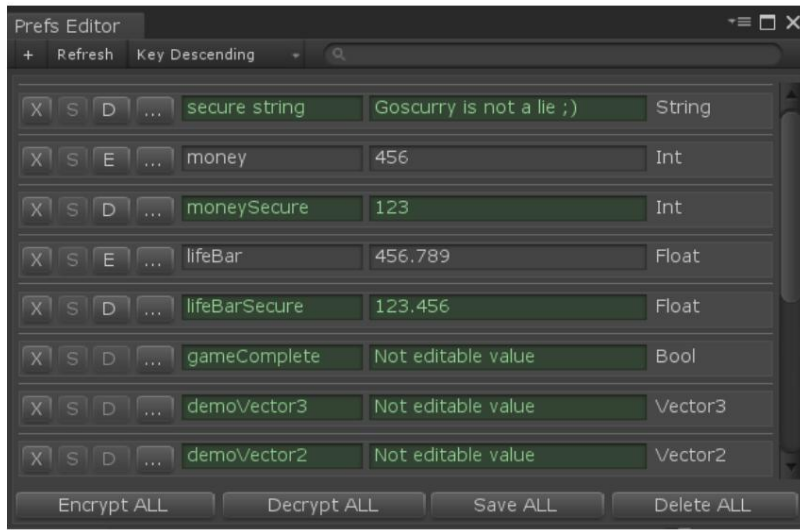
중요한:

- [OP](#)는 모든 데이터를 암호화하고 해독하므로 일반 [PP](#) 에 비해 느리게 작동한다는 점을 명심하세요 .
추가 리소스를 소비합니다. [성능 모호한 테스트](#)를 사용하여 직접 확인해 보세요.
[API 예제](#) 장면의 [PerformanceTests](#) 게임 개체 에 대한 구성 요소입니다 .
- 모든 기본 설정을 제거하려면 PlayerPrefs.DeleteAll () 대신 ObscuredPrefs.DeleteAll ()을 사용하여 내부를 적절하게 지우고 DeleteAll() 호출 후 가려진 새 기본 설정을 저장할 때 데이터 손실을 방지하세요.

ObscuredPrefs / PlayerPrefs 편집기 창

ACTk에는 유용한 편집기 창 – Prefs Editor가 포함되어 있습니다.
Unity Editor에서 PlayerPrefs 및 ObscuredPrefs를 편집하는 데 사용하세요.

메뉴 명령을 통해 엽니다.
[도구 > 코드 스테이지 > 치트 방지 툴킷 > 탭으로 Prefs 편집기](#) (유틸리티 창 모드도 사용 가능)



빠른 둘러보기:

- "+" 버튼: 새 기본 설정 추가 – 유형 선택, 선택적으로 암호화 활성화, 키 및 값 설정, 확인 버튼 누르기 • "새로 고침" 버튼: 모든 기본 설정 다시 읽기 및 목록 업데이트 • "키 내림차순" - 정렬 유형, 기타 가능 값: 키 오름차순, 유형, 모호함 • 검색 필드 – 입력을 시작하면 지정된 텍스트가 포함된 이름의 레코드가 필터링됩니다. • 페이지가 포함된 ObscuredPrefs 및 PlayerPrefs 목록(페이지당 50개의 레코드 표시) • 레코드 분석:

- "X" 버튼 – 기본 설정에서 기록을 삭제합니다. • "S" 버튼 – 기본 설정 저장소에 변경 사항을 저장합니다. • "E" / "D" 버튼 – 기본 설정 암호화/해독(모호한 기본 설정은 녹색으로 표시됨) • "..." 버튼 – 클립보드에 기본 설정 복사와 같은 추가 작업 • 키 및 값 필드를 사용하면 기본 설정의 키 또는 값 변경 가능(기본 설정에 편집 가능한 유형이 있는 경우) • 유형 라벨에 기본 설정 유형 표시 • 그룹 작업을 위한 하단의 일부 버튼(자명한 이름 포함) • 덮어쓰기 확인 기능이 있습니다. • 대량의 환경 설정(1k

이상)으로 작업하는 경우 환경 설정 구분 분석 진행 표시줄이 있습니다. • Win, Mac, Linux에서 작동합니다.

Prefs Editor는 Unity가 자신의 필요에 따라 사용하는 몇 가지 표준 환경 설정(예: UnitySelectMonitor, UnityGraphicsQuality 등)을 무시합니다.

Windows에서는 Editor와 독립 실행형 플레이어에 저장된 환경 설정이 서로 다른 위치에 배치되므로 Unity Editor 외부에 저장된 환경 설정에 대해서는 Prefs Editor를 사용할 수 없습니다.

일반 장치 잠금 기능 참고 사항

[ObscuredPrefs](#), [ObscuredFile](#) 또는 [ObscuredFilePrefs](#) 와 함께 장치 잠금 기능을 사용할 때 다음 사항을 고려하십시오.

- DeviceLockSettings 속성을 사용하면 저장된 데이터를 현재 장치에 잠글 수 있습니다. 이는 저장 게임이 한 장치에서 다른 장치로 이동하는 것을 방지하는 데 도움이 될 수 있습니다(예: 100% 게임 진행 상태로 저장하거나 구매한 게임 내 상품으로 저장).

경고: iOS에서는 위험을 무릅쓰고 사용하세요! iOS에서는 영구 장치 ID를 얻을 수 있는 안정적인 방법이 없습니다. 따라서 사용을 피하거나 DeviceIdHolder.DeviceId와 함께 사용하여 자체 장치 ID(예: 사용자 이메일)를 설정하세요.

세 가지 다른 수준의 데이터 잠금 엄격성을 지정할 수 있습니다. **없음:** 잠긴 데이터와

잠금 해제된 데이터를 모두 읽을 수 있으며 저장된 데이터는 잠금 해제된 상태로 유지됩니다.

소프트: 잠긴 데이터와 잠금 해제된 데이터를 모두 읽을 수 있지만 저장된 모든 데이터는 현재 장치에 고정됩니다.

엄격함: 현재 장치 데이터에 고정된 읽기만 가능합니다. 저장된 모든 데이터는 현재 장치에 고정됩니다.

[API 문서](#) 의 [DeviceLockSettings](#) 설명을 참조하세요. [더 많은 정보](#) 를 위해서.

Android 사용자: 이 기능을 사용하지 않는 경우 아래 문제 해결 섹션을 참조하세요.

- DataFromAnotherDeviceDetected 이벤트를 수신하여 다른 장치의 데이터를 감지합니다. ObscuredPrefs에서 호출 ObscuredFile \ ObscuredFilePrefs에서 세션당 한 번만 – 파일을 읽을 때마다 호출됩니다.

- [DeviceLockLevel.Strict](#)를 사용하면 기본적으로 다른 장치에서 저장한 내용을 읽을 수 없습니다. 줄일 수도 있습니다 [DeviceLockSettings.DeviceLockTamperingSensitivity](#) 를 낮춤 (감지 이벤트가 계속 발생함) 또는 [비활성화됨](#) (이벤트가 발생하지 않음)으로 설정합니다. 장치 ID가 예기치 않게 변경된 경우 데이터를 복원하는 데에도 사용할 수 있습니다.
- 장치 ID에 처음 액세스하면 일부 플랫폼에서 CPU 스파이크가 발생하여 데이터를 처음 로드하거나 저장할 때 게임 플레이에 문제가 발생할 수 있습니다. 이를 방지하려면 `DeviceIdHolder.ForceLockToDeviceInit()`를 호출하여 장치 잠금이 활성화된 상태에서 첫 번째 데이터 로드/저장 시 발생할 수 있는 급증을 방지하세요. 로딩 페이드 또는 스플래시 화면과 같은 정적 항목을 표시하는 동안 이 방법을 사용하면 원하는 시간에 장치 ID를 강제로 가져올 수 있습니다.
- `DeviceIdHolder.DeviceId` 속성을 사용하여 장치 ID를 명시적으로 설정합니다. 서버 측 인증이 있는 경우 고유한 사용자 ID(또는 이메일)에 대한 저장을 잠그는 데 유용할 수 있습니다. iOS에 가장 적합합니다. (iOS 7 이상에서는) 일관된 장치 ID를 얻을 수 있는 방법이 없기 때문에 우리가 가진 것은 공급업체 및 광고 ID뿐입니다. 두 가지 모두 변경할 수 있고 제한 사항이 있습니다.

코드 난독화 및 전반적인 코드 보호 참고 사항

ACTk는 소스 코드를 난독화하거나 보호하지 않으므로 모든 ACTk 기능을 사용하더라도 해커와 리버스 엔지니어링에 여전히 취약합니다.

따라서 가능하면 좋은 코드 난독 처리기와 IL2CPP 스크립팅 백엔드를 모두 사용하여 컴파일된 애플리케이션 코드를 분석하고 변경하는 것을 훨씬 어렵게 만드는 것이 좋습니다.

IL2CPP는 Mono의 IL 바이트 코드 대신 메타데이터를 사용하여 원시 바이너리 코드를 생성하므로 모든 IL 반전 도구를 쓸모 없게 만듭니다(예: [dnSpy](#) 등) 메소드 [본문을 제대로](#) 디컴파일하는 것이 훨씬 더 어려워집니다(비용이 많이 들고 사용하기 어려운 기본 디어셈블러 및 디컴파일을 통해서만 가능). 부가적인 보너스로 관리되는 어셈블리가 Mono ApplicationDomain에 삽입되는 것을 완전히 방지합니다.

메타데이터는 제한된 반사 목적으로 생성되며 메소드 코드 없이도 모든 네임스페이스, 클래스, 필드 등을 사용하여 IL 어셈블리를 구성할 수 있으며 사기꾼은 [IL2CPP Dumper](#) 와 같은 도구를 적극적으로 사용합니다. 전체 코드 구조를 살펴보기 위해 IL 어셈블리를 재구성합니다.

그렇기 때문에 IL2CPP를 빌드하기 전에 코드를 난독화하기 위해 이름 난독화 기능이 있고 Unity의 빌드 프로세스에 통합된 좋은 코드 난독처리기로 IL2CPP를 보완하는 것이 매우 중요합니다. 이러한 경우 대부분의 IL2CPP 메타데이터는 쓸모없는 이름으로 뒤죽박죽이 되어 재구성된 IL 어셈블리를 리버스 엔지니어링 및 분석하기가 매우 어려워집니다.

무료 코드를 포함하여 좋은 코드 난독 처리기가 많이 있지만 Unity와 제대로 통합된 코드는 많지 않습니다. 따라서 [Obfuscator](#)를 살펴보는 것이 좋습니다. Asset Store의 플러그인([타사](#) 섹션에서 이 플러그인에 대한 자세한 [내용을 읽어보세요](#)).

난독화되지 않은 이전 버전을 기반으로 새 바이너리 버전을 자동으로 난독화할 수 있는 도구가 있으므로 난독화된 초기 버전을 제공하는 것도 중요합니다.

특정 플랫폼 및 사용 사례에 대한 기본 보호기도 많이 있습니다. 가능하다면 기본 프로텍터 사용도 고려해 보세요. 보호 계층이 추가되어 애플리케이션을 편집하거나 리버스 엔지니어링하려는 해커 기술에 대한 요구 사항이 증가합니다(예: Denuvo, VMProtect 등).

아췌든, 클라이언트 측(즉, 해커가 접근할 수 없는 서버 측이 아닌) 코드의 민감한 부분이 있고 ACTk가 있는 경우 코드가 진짜이고 변조되지 않았는지 확인하는 것이 항상 좋은 생각입니다. 도와주려고 여기 있어요 [CodeHashGenerator](#).

Android 앱 설치 소스 검증

ACTk에는 Android 애플리케이션 설치 소스를 쉽게 파악할 수 있는 [AppInstallationSourceValidator](#) 도구가 있습니다.

[AppInstallationSourceValidator.IsInstalledFromGooglePlay\(\)](#)를 호출하여 Google Play 스토어에서 앱이 설치되었는지 확인하는 것만큼 사용이 간단합니다.

더 많은 제어가 필요한 경우 [AppInstallationSourceValidator.GetAppInstallationSource\(\)](#) API를 호출하여 설치 소스 패키지 이름과 해당하는 추측된 설치 소스 열거형 값을 수집할 수 있습니다.

```
// 이 줄을 .cs 파일 시작 부분에 바로 배치하세요!  
CodeStage.AntiCheat.Genuine.Android를 사용합니다.
```

```
// 설치 소스 가져오기 및 처리
```

```
var 소스 = AppInstallationSourceValidator.GetAppInstallationSource();

if (source.DetectedSource != AndroidAppSource.AccessError)
{
    Debug.Log($"설치 위치: {source.DetectedSource} (패키지 이름: {source.PackageName})");
    if (source.DetectedSource == AndroidAppSource.SamsungGalaxyStore)
        Debug.Log(" 갤럭시 스토어에서 앱이 설치되었습니다!");
}
또 다른
{
    Debug.LogError(" 설치 소스 검색에 실패했습니다!");
}
```

코드 무결성 검증

중요한:

- 현재까지는 Android 및 Windows PC 빌드만 지원됩니다.
- 주로 추가 코딩이 필요한 고급 Unity 개발자용

무결성 검증의 기본 아이디어는 컴파일된 바이너리의 고유한 지문(해시)을 만드는 것입니다. 이는 지문을 얻은 후 바이너리가 변경되는 즉시 변경되며 해당 지문을 런타임 애플리케이션의 현재 지문과 비교하는 것입니다.

초보 사기꾼을 처리해야 하는 경우 대부분의 간단한 경우에는 애플리케이션에서 바로 비교하는 것만으로도 충분하며 초보 사기꾼이 알아차리지 못할 가능성이 높으므로 간단한 해시 비교부터 자유롭게 시작할 수 있습니다. 애플리케이션의 C# 코드입니다.

그러나 위협의 특성으로 인해 고급 사기꾼이 검사를 변경할 수도 있으므로 C# 코드 외부에서 유효성 검사를 수행하는 것이 더 안전합니다. 여기서 가장 좋은 선택은 서버 측 유효성 검사를 수행하는 것입니다. 현재 해시를 서버로 보내고 처음에 생성된(허용 목록에 있는) 해시와 비교하여 코드가 변경되지 않았는지 확인하기만 하면 됩니다.

따라서 유효성 검사 절차는 일반적으로 정품 해시 가져오기, 실제 런타임 해시 가져오기, 해시 비교의 세 단계로 구성됩니다. 아래의 각 단계에 대한 자세한 내용을 읽어보세요.

중요한:

- 해시 생성 작업(편집기에서 또는 런타임 모두에서)은 빌드의 각 파일에 대해 파일별 해시를 생성하고 해당 파일의 요약 해시입니다 .
- 따라서 파일별 해시는 편집기(CodeHashGeneratorPostprocessor에서 가져옴) 와 런타임(CodeHashGenerator에서 가져옴) 모두에서 변경되지 않은 상태로 유지되어야 하지만 일부 경우에는 요약 해시가 다를 수 있습니다(예: 런타임 앱이 AAB 번들의 일부이고 내부에는 플랫폼별 파일이 적지 않습니다.)
- 권장 해시 비교 전략: 요약 해시를 먼저 비교하고 일치하지 않는 경우 사전 생성된 파일별 해시 허용 목록과 비교하여 모든 런타임 파일별 해시를 확인합니다(그리고 없는 파일을 무시하면서 알 수 없는 해시를 빌드 변경 트리거로 처리합니다) . .

CodeHashGenerator포스트 프로세서

이 편집기 스크립트를 사용하면 결과 빌드 파일에서 코드의 실제 해시를 생성할 수 있습니다.

[HashesGenerated](#) 이벤트를 수신하여 각 빌드 후에 결과 해시를 얻을 수 있습니다.

이 이벤트를 활성화하려면 [ACTk 설정](#) 에서 "[빌드 완료 시 코드 해시 생성](#)" 옵션을 활성화하세요. [결과 해시는 편집기 콘솔에도 인쇄됩니다](#)(이벤트를 구독하지 않은 경우에도 마찬가지).

내장된 메뉴 항목인 [도구 > 코드 스테이지 > 치트 방지 툴킷 > 외부 빌드 해시 계산을 통해 외부 빌드](#) 코드 해시를 수동으로 얻을 수도 있습니다.

또는 동일한 결과를 얻기 위해 Editor 코드에서 직접 `CodeHashGeneratorPostprocessor.CalculateExternalBuildHashes()` 를 호출할 수 있습니다.

이 포스트프로세서는 런타임에 빌드를 실행하지 않고 해시를 생성하도록 만들어졌습니다.

편집기 내 해시 생성에 문제가 발생하거나 Unity 빌드를 마친 후 코드를 사후 처리하는 경우 실제 빌드에서 [CodeHashGenerator](#)를 사용하여 런타임에 실제 해시를 생성하세요.

"예제/코드 정품 유효성 검사/스크립트/편집기"에 있는 편집기 PC 빌드 해싱 예제 대응 의 사용 예를 참조하세요 .

코드해시생성기

이 런타임 스크립트를 사용하면 실행 중인 앱에서 바로 해시를 생성할 수 있습니다.

해시를 반환하거나 정적 이벤트 HashGenerated를 구독하는 대기 가능한 생성Async()를 호출 하고 생성() 메서드를 호출하여 해시 생성을 시작합니다. 기본 해시 생성 코드는 플랫폼마다 다르지만 일반적으로 가능한 경우 별도의 스레드나 코루틴에서 작동하여 생성 프로세스를 원활하게 만들고 CPU 스파이크를 방지하여 전반적인 앱 성능 저하를 방지합니다.

신뢰할 수 있는 장치에서 런타임 시 첫 번째 정품 해시를 생성하여 나중에 사용자 장치의 런타임 해시와 비교하여 코드 무결성을 확인할 수 있습니다 . 하지만 Android App Bundle을 사용하는 경우 AAB가 Google Play에서 여러 APK로 분할되므로 기기에서 실행되는 앱에서 가져온 해시는 기기 하드웨어에 따라 다릅니다. 이러한 경우 가장 좋은 전략은 초기 aab 빌드에서 해시를 계산하여 모든 정품 파일 해시를 얻는 것입니다.

"예제/코드 정품 유효성 검사/스크립트/런타임" 및 "예제/코드 정품 유효성 검사/GenuineValidator" 장면에서 있는 런타임 PC 빌드 해싱 예제 대응의 사용 예를 참조하세요 .

중요한:

제공된 예는 개념을 보여주고 단순하게 유지하는 데 더 적합하기 때문에 Windows PC 플랫폼에만 해당됩니다.

일반적인 감지기 기능 및 설정

ACTk에는 다양한 치트를 감지하고 그에 따라 대응할 수 있는 다양한 감지기가 포함되어 있습니다. 모든 감지기에는 몇 가지 공통 기능과 설정 프로세스가 있습니다.

일반적으로 감지기를 설정하고 사용하는 세 가지 방법이 있습니다.

- Unity Editor를 통한 설정
- 코드를 통해 설정
- 혼합 모드

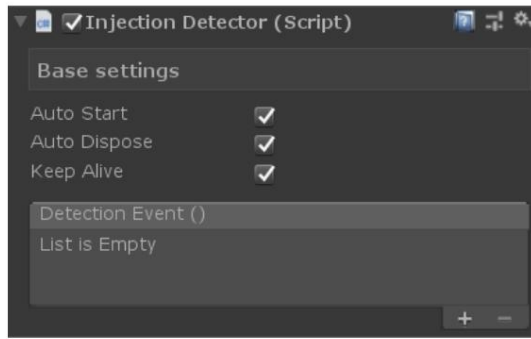
Unity Editor를 통한 설정

먼저 장면에 감지기를 추가해야 합니다. 이에 대한 두 가지 옵션이 있습니다.

1. **GameObject > Create Other > Code Stage > Anti-Cheat Toolkit** 메뉴를 사용하여 장면에 탐지기를 빠르게 추가합니다. 탐지기가 포함된 "**Anti-Cheat Toolkit Detectors**" 게임 개체가 장면에 생성됩니다(존재하지 않는 경우). 이는 감지기를 설정하는 데 권장되는 방법입니다.
2. 또는 다음을 통해 선택한 기존 게임 개체에 탐지기를 추가할 수 있습니다.
컴포넌트 > 코드 스테이지 > Anti-Cheat 툴킷 메뉴

다음 단계 – 추가된 감지기를 구성합니다. 모든 감지기에는 구성할 수 있는 몇 가지 공통 옵션이 있습니다.

예를 들어 새로 추가된 **주입 감지기**는 다음과 같습니다(일반적인 옵션 외에는 다른 옵션이 없으므로).



자동 시작: 장면 로드 후 감지기가 자동으로 시작되도록 활성화합니다. 그렇지 않으면 코드에서 명시적으로 시작해야 합니다(아래 세부정보 참조). 이 기능을 사용하려면 아래 설명된 감지 이벤트를 구성해야 합니다.

자동 폐기: 탐지기가 치트 탐지 후 자동으로 자체 폐기하고 자체 구성 요소를 파괴하도록 활성화합니다. 그렇지 않으면 감지기가 중지됩니다.

Keep Alive: 감지기가 새로운 레벨(장면) 로드를 유지하도록 활성화합니다. 그렇지 않으면 감지기가 새로운 수준의 부하에서 자체 처리됩니다.

감지 이벤트: 표준 [Unity 이벤트](#) 입니다. [치트 감지 시 실행되는](#) 감지기입니다.

권장 설정 - 모든 옵션을 활성화하고 감지에 적합한 이벤트를 설정합니다. 그러한 경우 항상 작동 중인 감지기가 장면을 통과하며 이동하며 감지 후 자체적으로 파괴됩니다. 감지기가 있는 동일한 게임 개체에 감지 콜백이 포함된 스크립트를 연결할 수 있으며 둘 다 장면 다시 로드에서 유지됩니다.

코드를 통한 설정

코드에서 설정하는 것을 선호하는 경우 코드의 어느 곳에서나 감지기의 정적 StartDetection(콜백) 메서드를 한 번 호출하면 해당 감지기가 기본 매개변수로 실행되도록 할 수 있습니다. 감지기가 존재하지 않는 경우 자동으로 장면에 추가됩니다.

옵션을 조정하려면 탐지기를 시작한 직후 정적 인스턴스 속성을 사용하여 구성 가능한 모든 필드와 속성에 접근하면 됩니다(장면에 해당 탐지기가 없으면 인스턴스는 null이 됩니다). 오버로드된 StartDetection() 메서드의 인수를 통해 일반적으로 초기 구성에 사용할 수 있는 감지기 옵션에 대한 일부 특정 항목은 초기 조정기에 사용할 수 있는 옵션을 파악하려면 감지기의 API 문서를 참조하세요.

주입 감지기 의 예는 다음과 같습니다 .

```
// 이 줄을 .cs 파일 시작 부분에 바로 배치하세요!  
CodeStage.AntiCheat.Detectors를 사용합니다 .
```

```
// 지정된 콜백으로 감지를 시작합니다.  
InjectionDetector.StartDetection(OnInjectionDetected);
```

```
// 이 메서드는 주입 감지 시 호출됩니다.  
private void OnInjectionDetected() { Debug.Log("알았어!"); }
```

혼합 설정

감지기를 사용하면 검사기의 구성과 코드를 통한 수동 실행을 결합하여 혼합 방식으로 설정할 수 있습니다.

Unity Editor 항목을 통해 설정 에 설명된 대로 장면에 감지기를 추가하여 감지를 유지할 수 있습니다. 이벤트가 비어 있고 코드 항목을 통한 설정 에 설명된 대로 StartDetection(콜백) 메서드를 사용하여 코드에서 수동으로 시작합니다 .

이 방법을 사용하면 감지기가 시작되어야 하는 순간을 제어할 수 있고 시작하기 전에 Inspector와 코드에서 인스턴스 속성 을 통해 감지기를 구성할 수 있습니다.

이러한 접근 방식을 사용하려면 감지기 검사기에서 자동 시작 옵션을 비활성화해야 합니다 .

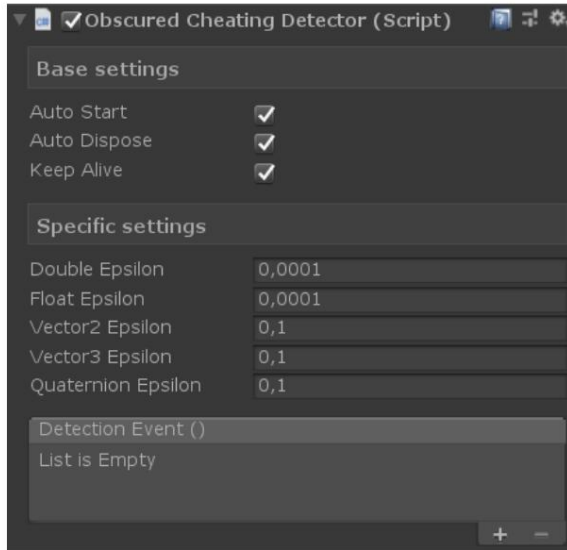
또한 StartDetection() 메서드(인수 없이)를 사용하여 검사기 및 시작 감지기에서 감지 이벤트를 채울 수도 있습니다 .

모호한 유형의 부정 행위 감지

Anti-Cheat Toolkit의 Obscured Cheating Detector를 사용 하면 [ObscuredPrefs](#)를 제외한 **모든** 가려진 유형 의 부정 행위를 감지할 수 있습니다 (다음 섹션에서 다루는 자체 감지 메커니즘이 있음).

실행 시 이 탐지기는 모호한 변수가 암호화되지 않은 가짜 값을 사기꾼을 위한 허니팟으로 사용할 수 있도록 허용합니다. 원하는 값을 찾을 수 있지만 값을 변경하거나 고정해도 부정 행위 감지를 트리거하는 것 외에는 아무 작업도 수행되지 않습니다.

모든 감지기의 설정 및 공통 기능에 대한 자세한 내용은 위의 일반적인 감지기 기능 및 [설정](#) 항목을 참조하세요 .



엠틀론: 치트 시도가 감지되기 전에 가짜와 실제 암호화 변수 사이에 허용되는 최대 차이입니다.

기본값은 대부분의 경우에 적합하지만 상황에 맞게 자유롭게 조정할 수 있습니다(예를 들어 어떤 이유로든 거짓 긍정이 있는 경우).

중요한:

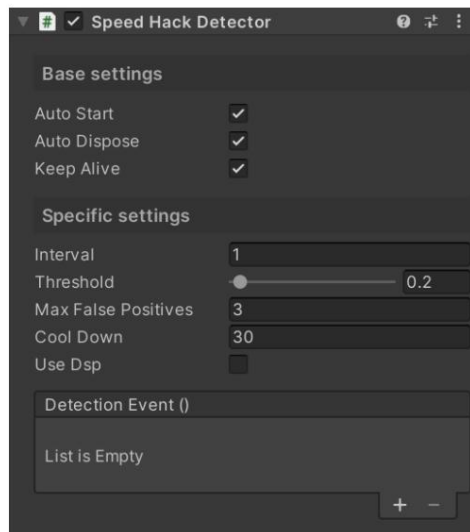
- 부정행위 탐지는 작동하지 않으며, 탐지기가 작동하지 않는 동안 가짜 암호화되지 않은 변수는 메모리에 존재하지 않습니다.
달리기. 따라서 [ObscuredCheatingDetector](#)를 사용하지 않거나 검사기에서 자동 실행을 비활성화하고 코드에서 실행하지 않으면 값이 메모리 검색기에 전혀 표시되지 않습니다.
- [ObscuredCheatingDetector.Instance.LastDetectionInfo](#) 속성을 사용하여 최신 탐지 이벤트에 대한 추가 컨텍스트를 얻을 수 있습니다 .

속도 해킹 감지 [\[비디오 튜토리얼\]](#)

이러한 유형의 부정행위는 사용이 간단하고 주변의 모든 어린이가 게임에서 시도할 수 있기 때문에 매우 인기가 있고 자주 사용됩니다. Speed Hack을 사용하면 몇 번의 간단한 클릭만으로 다양한 치팅 도구에서 게임 속도를 높이거나 낮출 수 있습니다.

Anti-Cheat Toolkit의 Speed Hack Detector를 사용하면 (Cheat Engine, Game Guardian 등과 같은 도구에서) 스피드 해킹 사용을 감지할 수 있습니다.

모든 감지기의 설정 및 공통 기능에 대한 자세한 내용은 위의 일반적인 감지기 기능 및 [설정](#) 항목을 참조하세요 .



간격: 감지 기간(초)입니다. 추가 오버헤드를 방지하려면 이 값을 1초 이상으로 유지하는 것이 좋습니다.

임계값: 속도를 높이거나 낮추는 데 허용되는 속도 승수입니다. 하드웨어 다양성으로 인해 타이머가 약간 빠르거나 느린 드문 경우에 대한 오탐 가능성을 줄이기 위해 도입되었습니다. 기본값 0.2는 1.2x 및 0.8x 게임 속도를 모두 허용하며 일반적으로 충분히 안정적이며 대부분의 알려진 타이머 문제에서 작동합니다.

문제.

최대 거짓 긍정: 매우 드문 경우(시스템 시계 오류, OS 결함 등) 감지기가 거짓 긍정을 생성할 수 있습니다. 이 값을 사용하면 치트에 대응하기 전에 지정된 양의 스피드 해킹 감지를 건너뛸 수 있습니다. 실제 속도 해킹이 적용되면 감지기는 모든 검사에서 이를 감지합니다. 빠른 예: 간격을 1로 설정하고 최대 거짓 긍정을 5로 설정했습니다. 애플리케이션에 속도 해킹이 적용된 경우 감지기는 그 후 ~5초 후에 감지 이벤트를 실행합니다(간격 * 최대 거짓 긍정 + 다음까지 남은 시간) 확인하다).

쿨다운: 연속적으로 지정된 횟수의 성공적인 샷 이후 내부 오탐지 카운터를 재설정할 수 있습니다.

앱에 드물지만 주기적인 잘못된 감지가 있는 경우(예를 들어 지속적인 OS 타이머 문제가 있는 경우) 유용할 수 있습니다.

오탐지 카운터가 서서히 증가하여 허위 속도 해킹 탐지가 발생합니다.

냉각 기능을 완전히 비활성화하려면 값을 0으로 설정하십시오.

DSP 사용: 샌드박스 환경(예: WebGL, 속도 해킹이 내장된 VM 등)에서 속도 해킹을 포착할 수 있는 DSP 오디오 모듈을 활성화합니다. 기본적으로 꺼져 있습니다.

경고: 민감도가 높기 때문에 일부 하드웨어, 특히 iOS에서 실행할 때 오탐지가 발생할 수 있습니다.

항상 대상 장치에서 오탐지 여부를 테스트하고 위험을 감수하고 사용하세요!

참고를 위해 "내부"에서 일어나는 일에 대한 몇 가지 예를 보여 드리겠습니다. 다음 매개변수를 사용하겠습니다.

간격 = 1, 최대 거짓양성 = 5, 쿨다운 = 30

전. 1: 속도 해킹 없이 애플리케이션이 작동합니다. 감지기는 매초마다 내부 점검을 실행합니다 (간격 == 1). 문제가 발생하여 타이머가 비동기화되면 거짓 긍정 카운터가 1씩 증가합니다. 30초 후 (OS 중단 없이) 원활한 작업의 (간격 * 쿨다운) 오탐지 카운터는 다시 0으로 설정됩니다. 이는 원치 않는 검색을 방지합니다.

전. 2: 애플리케이션이 시작되고 잠시 후 Speed Hack이 애플리케이션에 적용되었습니다. 감지기는 매초 내부 검사를 실행하여 오탐지 카운터를 1씩 올립니다. 5초 (간격 * 최대 오탐지율) 후에 치트가 감지됩니다.

치터가 탐지를 피하려고 시도하고 3초 후에 스피드 해킹을 중지하면 다시 치트를 적용하기 전에 30초를 더 기다려야 합니다. 꽤 짜증나네요. 그리고 Cool Down을 60(대기 1분)까지 늘리면 더욱 짜증날 수 있습니다. 그리고 부정행위를 하는 사람은 Cool Down을 활용하려면 Cool Down에 대해 전혀 알아야 합니다.

SpeedHackProofTime API

[SpeedHackDetector](#)를 사용하면 [SpeedHackProofTime](#) 클래스를 활용하여 Unity의 시간 대신 안정적인 타이머를 사용할 수 있습니다.* 일반적으로 속도 해킹으로 인해 어려움을 겪는 타이머. Mimics [Time.*](#) API(고정* 물리 API 제외).

[SpeedHackDetector](#)가 실행 중일 때만 작동하며 실제 속도 해킹이 감지될 때까지 Unity의 타이머로 대체됩니다.

애니메이션 및 내부 시간 관련 계산이 감지된 속도 해킹에 영향을 받지 않도록 만드는 데 사용됩니다.

"[Examples/API 예제/Scripts/Runtime/InfiniteRotatorReliable](#)" 스크립트 및 "[API 예제](#)" 장면에서 사용 예를 참조하세요.

중요한:

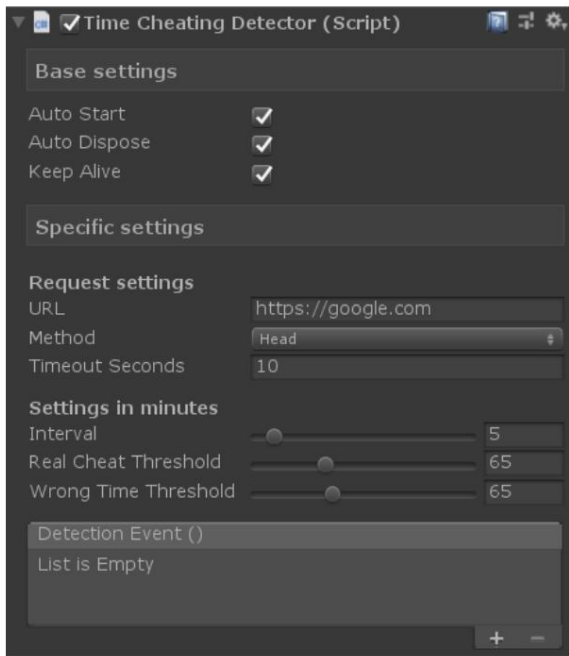
- [SpeedHackProofTime](#)은 어떤 경우에는 작동하지 않을 수 있습니다(예: 신뢰할 수 있는 보안에 접근할 수 있는 법적 방법이 없는 경우). 타이머(Chrome 브라우저의 프로세스와 같은 샌드박스 프로세스에서 발생할 수 있음)

시간 부정 행위 감지

플레이어는 종종 이러한 유형의 부정 행위를 사용하여 건물 진행, 몇 시간 또는 며칠에 걸친 에너지 냉각 및 복원과 같은 게임의 일부 장기 프로세스를 가속화합니다. 모든 부정행위자는 그러한 부정행위를 활용하기 위해 해야 할 일이 있습니다. 시스템 시간을 변경하여 게임이 새 항목을 만들기 시작한 지 하루 또는 몇 시간이 지났는지 "생각"하도록 하십시오.

Anti-Cheat Toolkit의 시간 부정 행위 감지기를 사용하면 온라인 시간 서버를 사용하여 이러한 부정 행위를 감지할 수 있습니다. 이 탐지기를 사용하려면 인터넷 연결이 필요합니다.

모든 감지기의 설정 및 공통 기능에 대한 자세한 내용은 위의 일반적인 감지기 기능 및 설정 항목을 참조하세요 .



URL: HEAD 또는 GET 요청에 올바른 날짜 응답 헤더 값을 반환하는 리소스에 대한 절대 URL입니다.
WebGL에서 실행할 때 CORS 제한을 피하기 위해 필요한 경우 URL이 현재 도메인으로 자동 변경되었습니다.

Method : 사용할 메소드를 요청합니다. Head는 더 빠르고 트래픽을 덜 사용하지만 일부 웹 서버는 HEAD 요청을 너무 자주 봇 활동으로 처리하여 클라이언트를 일시적으로 차단할 수 있습니다. 문제가 있는 경우 호환성이 더 높은 Get 메서드를 사용하세요.

시간 초과 초: 요청을 중단하기 전에 서버 응답을 기다리는 시간입니다.

간격: 탐지 기간(분)입니다. 트래픽과 추가 리소스 사용량을 줄이려면 너무 짧은 간격(1분 미만)을 사용하지 마십시오.

실제 치트 임계값: 온라인 및 오프라인 시간 차이에 대한 두 후속 측정 간에 허용되는 최대 차이(분)입니다. 차이가 이 값을 초과할 때 실제 치트가 등록됩니다.

잘못된 시간 임계값: 온라인 시간과 오프라인 시간 사이에 허용되는 최대 차이입니다. 차이가 이 값을 초과하면 확인 결과와 함께 콜백 또는 이벤트가 발생합니다(스크립팅에서만 구독 가능): [TimeCheatingDetector.CheckResult.WrongTimeDetected](#)

중요한:

- 추가 이벤트인 CheatChecked를 구독하면 부정행위 확인 결과에 대한 전체 정보를 얻을 수 있습니다.
감지를 시작할 때 스크립팅을 통해 설정할 수 있는 모든 콜백에 동일한 대리자가 사용됩니다.
- 부정 행위를 확인하는 동안 인터넷이 연결되지 않으면 [CheckResult.Error](#) 와 함께 CheatChecked 이벤트가 발생합니다.
결과 인수 및 ErrorKind.OnlineTimeError 오류 인수를 사용하면 검사를 건너뛰고 지정된 간격으로 다시 재시도하도록 예약됩니다. 감지기는 인터넷 연결 상태가 좋지 않거나 가끔씩 연결되는 장치에서 제대로 작동합니다.

- 이 감지기에는 인터넷이 필요하므로 Android의 [인터넷](#) 권한(일반적으로 자동 부여됨) 과 같은 모바일 플랫폼에 대한 추가 권한 요청을 생성합니다 . 이 감지기가 필요하지 않고 추가 권한을 제거하려면 [ACTk 설정](#) 창 에서 [ACTK_PREVENT_INTERNET_PERMISSION](#)을 확인하세요 .
- 현지 UTC 시간과 온라인 UTC 시간 간의 차이로 이어지는 수동 시스템 시간 변경은 적용되지 않습니다.
부정 행위로 처리되지만 대신 잘못된 시간으로 처리됩니다.
- ForceCheck* 방법을 사용하여 부정 행위 검사를 수동으로 실행하고 결과를 얻을 수 있습니다(이 방법에 대한 API 문서 참조).
사용 예를 보려면). 원하는 순간에 수동으로 점검을 실행할 때 감지기를 완전 수동 모드로 전환하기 위해 0 간격과 결합될 수 있습니다.

Wallhack 감지 [\[비디오 튜토리얼\]](#)

종종 "월 해킹"이라고 언급되는 몇 가지 다른 부정 행위 방법이 있습니다. 플레이어는 투명하거나 와이어프레임 벽을 통해 볼 수 있고, 유령처럼 벽을 통과할 수 있으며, 벽을 통해 총을 쏠 수 있습니다.

ACTk에는 이 모든 것을 포괄하는 WallHack Detector가 있습니다.

몇 가지 서로 다른 모듈로 구성되어 있으며, 각 모듈은 서로 다른 종류의 치트를 감지합니다.

실행하는 동안 WallHack Detector는 장면에 서비스 컨테이너 "[WH Detector Service]"를 생성하고 이를 3x3x3 큐브 내의 가상 샌드박스로 사용하여 설정에 따라 다양한 항목을 생성합니다.

모든 감지기의 설정 및 공통 기능에 대한 자세한 내용은 위의 일반적인 감지기 기능 및 설정 항목을 참조하세요 .



Rigidbody: Rigidbody 해킹을 통해 만들어진 "벽 통과" 종류의 치트를 확인하려면 이 모듈을 활성화하십시오. 캐릭터에 Rigidbody를 사용하지 않는 경우 일부 리소스를 절약하려면 비활성화하세요.

캐릭터 컨트롤러: 이 모듈을 활성화하면 캐릭터 컨트롤러 해킹을 통해 만들어진 "벽 통과" 종류의 치트를 확인할 수 있습니다. 캐릭터 컨트롤러를 사용하지 않는 경우 일부 리소스를 절약하려면 비활성화하세요.

와이어프레임: 이 모듈을 활성화하면 셰이더 또는 드라이버 해킹을 통해 만들어진 "벽을 통해 볼 수 있는" 종류의 치트(벽이 와이어프레임, 알파 투명 등으로 변경됨)를 확인할 수 있습니다. 걱정하지 않을 경우를 대비해 일부 리소스를 저장하지 않도록 설정하세요.
그런 사기꾼.

참고: 런타임에 제대로 작동하려면 특정 셰이더가 필요합니다. 아래에서 자세한 내용을 읽어보세요.

와이어프레임 자연: 와이어프레임 모듈 검사 사이의 시간(초)은 1초에서 최대 60초입니다.

Raycast: 이 모듈을 활성화하면 Raycast 해킹을 통해 만들어진 "벽을 통과하는" 유형의 치트를 확인할 수 있습니다. 그러한 치트에 대해 걱정하지 않을 경우를 대비해 일부 리소스를 저장하지 않도록 설정하세요.

Raycast Delay: Raycast 모듈 검사 사이의 시간(초)은 1초에서 최대 60초입니다.

스폰 위치: 장면에서 3x3x3 빨간색 와이어프레임 큐브로 표시되는 동적 서비스 컨테이너의 세계 좌표입니다(탐지기가 있는 Game Object를 선택할 때 표시됨).

Max False Positives: 실제 탐지 전에 연속 탐지가 허용됩니다. 각 모듈에는 자체 감지 카운터가 있습니다. 그것

치트가 감지될 때마다 증가하고 해당 모듈의 첫 번째 성공 시 재설정됩니다(치트가 감지되지 않은 경우). 그러한 카운터 중 하나라도 Max False Positives 값보다 커지면 탐지는 최종 실제 탐지를 등록합니다.

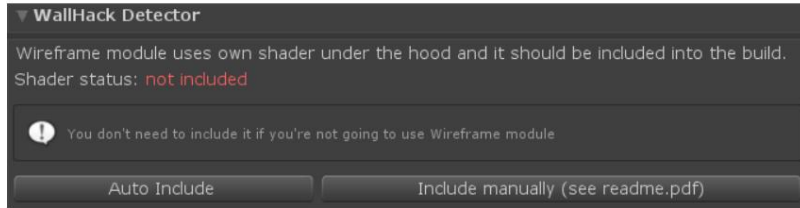
와이어프레임 모듈 셰이더 설정

Wireframe 모듈은 내부적으로 **Hidden/ACTk/WallHackTexture** 셰이더를 사용합니다. 따라서 이러한 셰이더는 런타임에 존재하도록 빌드에 포함되어야 합니다.

Hidden/ACTk/WallHackTexture 셰이더가 포함 되어 있지 않은 경우 런타임 시 로그에 오류가 표시되며 , 셰이더를 포함하지 않고

WallhackDetector를 실행할 때 이를 포함하라는 메시지가 편집기에 표시됩니다.

ACTk 설정 창을 통해 셰이더를 쉽게 추가하거나 제거할 수 있습니다. _____



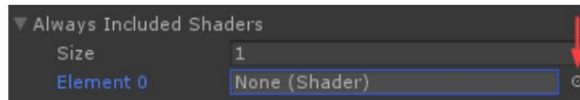
Hidden/ACTk/WallHackTexture 셰이더를 **항상 포함된** 셰이더 목록 에 자동으로 추가하려면 **"자동 포함"** 버튼을 누르세요 . _____

두 번째 버튼 ("**수동 포함**") 을 눌러 프로젝트의 그래픽 설정을 열고 셰이더를 추가할 수도 있습니다.

항상 포함된 셰이더 목록에 수동으로 추가합니다.

목록에 셰이더를 수동으로 추가하려면:

- 목록에 요소를 하나 더 추가합니다.
- 비어 있는 새 요소 옆에 있는 전구를 클릭합니다(아래 이미지의 빨간색 화살표 참조).

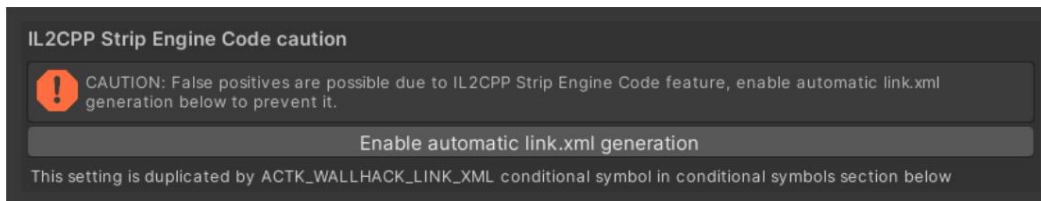


- 열린 창에서 "wallhack"을 검색하고 두 번 클릭하여 **WallHackTexture**를 선택합니다.

이것이 와이어프레임 모듈 설정의 전부입니다.

WallHack 감지기 및 IL2CPP 공지

IL2CPP로 빌드할 때 Unity 엔진 스트리핑이 시작되어 사용되지 않는 구성요소를 제거합니다. 이로 인해 때때로 WallHack Detector에서 사용되는 항목이 제거되어 오탐지가 발생할 수 있습니다. 이를 방지하려면 **ACTk 설정** (WallHack Detector 섹션 아래) 에서 자동 link.xml 생성을 활성화해야 합니다 .



중요한:

- Wallhack은 일반적으로 데스크톱 FPS 게임에 적용되는 매우 특정한 유형의 치트이므로 게임이 문제를 겪을 수 있는지 확인하세요. 감지에는 상당한 양의 추가 리소스가 필요하기 때문에 감지기를 사용하기 전에 먼저 확인하세요.
- **ACTk 설정** 에서 **ACTK_WALLHACK_DEBUG** 조건부 컴파일 기호를 활성화 하여 _____ 서비스 개체의 렌더러를 확인하고 게임에서 확인하세요. 또한 와이어프레임 모듈의 결과 텍스처에 대한 OnGUI 출력도 가능합니다. 이 기호는 개발 빌드 또는 Editor에서만 작동합니다.
- 서비스 컨테이너 내의 모든 개체는 "Raycast 무시" 레이어에 배치되며 다음을 통해 렌더러를 비활성화합니다. 기본.
- 3x3x3 서비스의 충돌을 피하기 위해 Spawn Position을 비어 있고 격리된 공간으로 설정하는 것이 중요합니다. 개체와 함께 샌드박스를 사용하여 오탐지와 개체의 오작동을 초래합니다.

- 감지기가 지속적으로 물체를 생성 및 이동하고, 물리학을 사용하고, 다른 물체를 생성할 수 있다는 점을 염두에 두는 것도 중요합니다. 설정에 따라 실행하는 동안 샌드박스에서 계산합니다.

관리형 DLL 주입 감지 [\[비디오 자습서\]](#)

중요 #1: Mono Android 및 Mono PC 빌드에서만 작동합니다!

중요 #2: IL2CPP 빌드에서는 어셈블리 주입이 불가능합니다. 전용 [코드 보호 섹션에서 자세한 내용을 읽어보세요.](#)

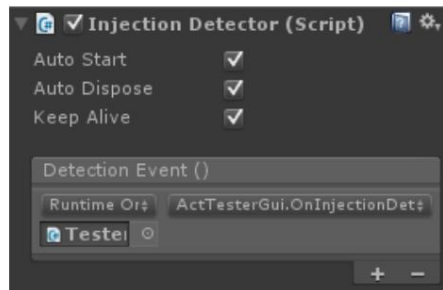
중요 #3: 오탐지를 유발하는 특정 어셈블리로 인해 편집기에서 비활성화되었습니다. [ACTK_INJECTION_DEBUG](#) 기호를 사용하여 편집기에서 강제로 실행하세요.

경고: 대상 장치에서 앱을 테스트하고 주입 감지기에서 오탐지가 없는지 확인하십시오. 이러한 문제가 있는 경우 감지된 어셈블리를 화이트리스트(아래 설명)에 추가해 보십시오.

이러한 종류의 부정 행위는 다른 부정 행위만큼 널리 사용되지는 않지만 여전히 Unity 앱에 사용되므로 무시할 수 없습니다. 이러한 주입을 구현하려면 치터에게는 고급 기술이 필요하므로 대부분은 특수 포털에서 숙련된 사람들로부터 치트 주입기를 구입합니다. 무언가를 주입하는 가장 쉬운 방법 중 하나 - [단일 조립 주입기](#) 사용 또는 유사한 소프트웨어. 일부 미친 사람들은 Cheat Engine의 Auto Assemble을 [사용하기도 합니다](#). [치트 엔진이 얼마나 멋진지](#) 아시나요?

Anti-Cheat Toolkit의 주입 감지기를 사용 하면 관리되는 어셈블리(DLL)를 앱에 주입할 때 반응할 수 있습니다. 런타임에서 사용하기 전에 [ACTk 설정](#) 에서 활성화해야 합니다 ([모노 주입 감지기 지원](#) 확인란 추가).

모든 감지기의 설정 및 공통 기능에 대한 자세한 내용은 위의 일반적인 감지기 기능 및 설정 항목을 참조하세요 .



이 감지기는 일반적인 옵션을 제외하고는 조정할 수 있는 추가 옵션이 없습니다(참고: 코드에서 시작할 때 문자열 인수를 허용하는 콜백을 전달합니다 - 감지 지원).

고급 치트를 탐지하는 간단한 방법!

주입 감지기 내부(고급)

여러분에게 통찰력을 제공하기 위해 중요한 주입 감지기 "내부" 주제에 대해 몇 마디 말씀드리겠습니다.

일반적으로 모든 Unity 앱은 세 가지 어셈블리 그룹을 사용할 수 있습니다.

- 시스템 어셈블리: System.Core, mscorlib, UnityEngine 등.
- 사용자 어셈블리: DOTween.dll(훌륭한 [DOTween](#) 의 어셈블리)과 같은 타사 어셈블리를 포함하여 프로젝트에서 사용할 수 있는 모든 어셈블리입니다. 트위닝 라이브러리) 또는 [외부](#) 패키지의 어셈블리 + 사용자 코드에서 Unity가 생성한 어셈블리(Assembly-CSharp.dll, 어셈블리 정의 어셈블리 등)
- 런타임 생성 및 외부 어셈블리. 일부 어셈블리는 리플렉션을 사용하여 런타임에 생성되거나 로드될 수 있습니다. 외부 소스(예: 웹 서버)에서. 이는 일반적으로 드문 경우입니다.

주입 감지기는 앱에서 외부 어셈블리를 감지하기 위해 신뢰할 수 있는 모든 유효한 어셈블리에 대해 알아야 합니다.

자동으로 처음 두 그룹을 다룹니다. 마지막 그룹은 수동으로 다루어야 합니다(아래 참조).

감지기는 허용된 어셈블리를 건너뛰기 위해 화이트리스트 접근 방식을 활용합니다. 애플리케이션을 빌드할 때 이러한 목록을 자동으로 생성하고 이를 빌드에 포함합니다(ACTk 설정 창에서 [주입 탐지기 활성화](#) 확인란을 선택한 경우).

이 화이트리스트는 두 부분으로 구성됩니다.

- 프로젝트에 사용된 어셈블리의 동적 화이트리스트(첫 번째 및 두 번째 그룹 포함)
- 사용자 정의 화이트리스트(세 번째 그룹, 아래 세부정보 참조).

대부분의 경우 아무것도 하지 말고 설정에서 주입 탐지기 지원을 활성화하고 장면에서 설정하면 됩니다.

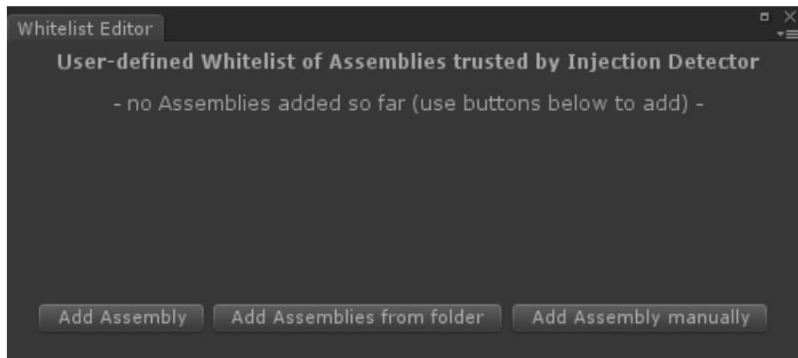
제대로 작동하도록 하는 코드입니다.

그러나 프로젝트가 외부 소스에서 일부 어셈블리를 로드하거나(이러한 어셈블리가 프로젝트 자산에 존재하지 않음) 런타임에 어셈블리를 생성하는 경우 거짓양성을 방지하기 위해 이러한 어셈블리에 대해 감지기에 알려야 합니다. 이러한 이유로 사용자 정의 화이트리스트 편집기가 구현되었습니다.

사용자 정의 화이트리스트를 채우는 방법

화이트리스트에 어셈블리를 추가하려면 다음의 간단한 단계를 따르십시오.

- "[도구](#) > [코드 단계](#) > [Anti-Cheat 툴킷](#) > [삽입 탐지기 화이트리스트 편집기](#)" 메뉴를 사용하여 화이트리스트 편집기를 열거나 설정 창의 "[화이트리스트 편집](#)" 버튼:



- 세 가지 가능한 옵션을 사용하여 새 어셈블리를 추가합니다. 단일 파일("어셈블리 추가"), 반복 폴더 검색("어셈블리 추가") 폴더에서"), 수동 추가 - 전체 어셈블리 이름 입력("수동으로 어셈블리 추가").

그게 다야!

감지된 어셈블리를 수동으로 추가하고 전체 이름을 모르는 경우 주입 감지기에서 디버그를 활성화하세요. [ACTk 설정](#) 창에서 [ACTK_INJECTION_DEBUG](#)를 확인하고 감지기가 어셈블리를 포착하도록 하세요. 콘솔 로그에서 "[ACTk] [주입된 어셈블리 발견.](#)" 문자열 뒤에 전체 이름이 표시됩니다. 이 기호는 개발 빌드 또는 Editor에서만 작동합니다.

화이트리스트 편집기를 사용하면 목록에서 어셈블리를 하나씩 제거하고(어셈블리 이름 옆에 작은 "-" 버튼 있음) 전체 화이트리스트도 지울 수 있습니다.

사용자 정의 화이트리스트는 다른 모든 설정과 함께 [ProjectSettings/ACTkSettings.asset](#) 파일에 저장됩니다.

중요한:

- 원하는 경우 삽입 예시에 대한 송장 ID를 이메일로 보내주세요(이 문서 끝부분에 있는 [자원 연락처 참조](#)). 실제로 [InjectionDetector](#)를 사용해 보세요 .
- [ACTk 설정](#) 창에서 [ACTK_INJECTION_DEBUG_VERBOSE](#) 및 [ACTK_INJECTION_DEBUG_PARANOID](#) 옵션을 확인하여 추가 디버그 정보에 대해 이러한 [컴파일 기호](#)를 [활성화](#)할 수 있습니다. 이러한 기호는 개발 빌드 또는 Editor에서만 작동합니다.

타사 플러그인 통합 및 메모

중요한:

ACTk를 지원하는 일부 타사 플러그인의 경우 프로젝트에 [ACTK_IS_HERE](#) 조건부 기호가 필요할 수 있습니다. [ACTk 설정](#)에서 활성화할 수 있습니다. _____

난독 처리기

Anti-Cheat Toolkit은 부정 행위를 방지하고 탐지하는 다양한 방법을 제공합니다.

또한 설정을 완료하려면 우수한 코드 보호 기능으로 ACTk를 보완하는 것이 좋습니다.

코드 리버스 엔지니어링의 난이도를 높이기 위해 제가 제안하는 두 가지 첫 번째 단계는 다음과 같습니다.

- 가능하다면 Mono 대신 IL2CPP를 사용하는 것이 좋습니다. 이렇게 하면 부정 행위자가 코드를 검사하고 분석하기가 더 어려워집니다. IL2CPP 빌드에는 사기꾼이 일반적으로 ILSpy 등에서 디컴파일하는 IL 바이트 코드가 없습니다. 그렇지만

부정 행위자는 여전히 악의적으로 사용될 수 있는 모든 코드(네임스페이스, 클래스 이름, 메서드 이름 등)의 메타데이터를 보유하고 있습니다. 이것이 두 번째 단계가 존재하는 이유입니다.

- 난독처리를 사용하여 클래스 이름, 메소드, 필드 등을 사기꾼에게 의미 없는 혼란으로 만드는 것을 고려하십시오. 그럴 것이다 코드 리버스 엔지니어링의 난이도를 극적으로 높입니다.

다행히도 상점에는 격차를 메우고 코드를 쉽고 원활하게 보호할 수 있는 유용한 자산이 있습니다. [Obfuscator](#), 갑자기 =) Unity용으로 제작되었기 때문에 Unity 이벤트, 메시지, 코루틴 등에 대해 "알고" 있으므로 [ACTK_EXCLUDE_OBFUSCATION](#) 기호를 적용할 필요가 없습니다 (아래 이 기호에 대한 자세한 내용 참조).

모든 어셈블리 정의의 난독화 옵션을 사용하거나 난독화기 옵션의 어셈블리 목록에 [ACTk.Runtime.dll](#)을 추가하여 ACTk의 내부도 난독화해야 합니다.

플레이메이커

현재 ACTk는 [PlayMaker](#)를 부분적으로 지원합니다. (오후), [패키지에서 사용할 수 있는 PM 작업은 거의 없습니다.](#)

[통합/PlayMaker.unity패키지](#). PM Actions Browser에 새 작업을 추가하려면 프로젝트로 가져오세요.

[Scripts/PlayMaker/Examples](#) 폴더에서 몇 가지 예를 확인하세요.

[ObscuredPrefs](#) 및 [ObscuredFilePrefs](#)는 완벽하게 지원됩니다. Actions Browser의 [PlayerPrefs](#) 섹션에서 [Obscured](#) * 액션을 찾을 수 있습니다.

- [ObscuredCheatingDetector](#)를 제외한 모든 탐지기가 완벽하게 지원됩니다. [Anti-Cheat](#)에서 탐지기 작업을 찾을 수 있습니다. Actions Browser의 [툴킷](#) 섹션.
- PM에 대한 기본 Obscured 유형은 현재 지원되지 않습니다. PM은 새로운 변수 유형 추가를 허용하지 않습니다. 하지만 그것은 손으로 통합하는 것이 가능합니다. [예가 있습니다](#) 그리고 [설명 크라이슈바이데\(kreischweide\)](#)

행동 디자이너

현재 ACTk는 [Opsive Behavior Designer](#)를 완벽하게 지원합니다. (BD). BD 작업이 거의 없습니다(액션 및 조건부).

패키지에서 사용 가능한 SharedVariables:

[통합/BehaviorDesigner.unity패키지](#). Anti-Cheat Toolkit을 BD와 통합하려면 프로젝트로 가져오세요.

[Scripts/BehaviorDesigner/Examples](#) 폴더에서 몇 가지 예를 참조하세요.

ACTk와 잘 작동하는 기타 타사 플러그인

- Lovatto Studio MFPS [치트 방지 및 보고 애드온](#).
- [간편한 IAP 시스템](#) 및 [IAP 영수증 유효성 검사기](#) IAP 영수증을 처리하고 검증하기 위한 FLOBUK 제공
- [크로스 플랫폼 네이티브 플러그인](#) 클라우드에 저장하고 순위표 등을 설정하는 Voxel Busters Interactive 제공
- Stan의 [Android 기본 플러그인](#).

여기에서 플러그인을 보고 싶다면 알려주시기 바랍니다.

문제 해결

- 업데이트 후 오류가 발생했습니다.
업데이트 문제를 방지하려면 이전 버전(전체 [CodeStage/AntiCheatToolkit](#))을 완전히 제거하십시오.
폴더) 업데이트된 ACTk 패키지를 프로젝트로 가져오기 전에.
- 감지기 중 하나에 대해 거짓 긍정 또는 기타 오작동이 있습니다.
코드를 통해 기존 장면 감지기를 시작하는 경우 [Awake](#) 또는 OnEnable 단계가 아닌 MonoBehaviour의 Start() 단계에서 StartDetection() 메서드를 사용하고 있는지 확인하세요. [ACTk 설정](#) 창에서 [ACTK_DETECTION_BACKLOGS](#) 플래그를 활성화하면 InjectDetector를 제외한 모든 감지기(원인이 있는 콜백이 있으므로)는 감지에 대한 세부 정보를 인쇄합니다. 가능하다면 이 플래그를 활성화하여 개발 빌드에서 생성된 로그에 대한 오탐지를 보고해 주세요.

- [InjectionDetector](#)에 잘못된 긍정이 있습니다 .

게임에서 사용하는 모든 외부 라이브러리를 화이트리스트에 추가했는지 확인하세요(메뉴: [도구 > 코드 스테이지 > 치트 방지](#))

[툴킷 > 주입 감지기 화이트리스트 편집기](#)). 이 화이트리스트를 채우는 방법은 위의 사용자 정의 화이트리스트 섹션을 채우는 방법을 참조하세요 . 도움이 되지 않으면 저에게 연락해주세요.

- [WallHackDetector](#) 오탐지가 있습니다 .

감지기의 생성 위치를 올바르게 구성 했는지, 감지기의 서비스 개체가 게임의 개체와 교차하지 않는지 확인하세요 . 또한 편집 > 프로젝트 설정 > 물리의 레이어 충돌 매트릭스에서 Ignore Raycast 레이어가 자체적으로 충돌하는지 확인하세요 . [WallHackDetector](#)는 모든 서비스 개체를 Ignore Raycast 레이어에 배치하여 게임 개체와의 불필요한 충돌을 방지하고 이러한 개체를 서로 충돌시키는 데 도움을 줍니다. 그렇기 때문에 Ignore Raycast 레이어가 자체적으로 충돌하는지 확인해야 합니다.

- [ObscuredCheatingDetector](#) 오탐지가 있습니다 .

직렬화된 모호한 유형의 손상된 인스턴스가 없는지 확인하십시오. 그렇게 하려면 다음 메뉴로 이동하십시오: [도구 > 코드 단계 > 치트 방지 툴킷 > 유효성 검사](#)

- [WallHackDetector](#) 에서 '클래스 *가 존재하지 않기 때문에 구성 요소를 추가할 수 없습니다.' 오류가 발생합니다 .

WallHackDetector는 BoxCollider, MeshCollider, CapsuleCollider, Camera, Rigidbody, CharacterController, MeshRenderer와 같은 Unity 구성 요소를 사용합니다. 이러한 오류는 해당 구성 요소 중 하나가 제거될 때 나타날 수 있습니다. 예:

IL2CPP [스트립 엔진 코드 별 옵션](#). 구성 요소가 제거되는 것을 방지하려면 빌드에 있는 모든 장면의 게임 개체에 해당 구성 요소를 추가하거나 이러한 link.xml 파일을 자산 폴더의 아무 곳이나 넣을 수 있습니다.

```
<link>
  <어셈블리 전체 이름="UnityEngine">
    <type fullname="UnityEngine.BoxCollider" 보존="all"/>
    <type fullname="UnityEngine.MeshCollider" 보존="all"/>
    <type fullname="UnityEngine.CapsuleCollider" 보존="all"/>
    <type fullname="UnityEngine.Camera" presents="all"/>
    <type fullname="UnityEngine.Rigidbody" 보존="all"/>
    <type fullname="UnityEngine.MeshRenderer" 보존="all"/>
    <type fullname="UnityEngine.CharacterController" 보존="all"/>
  </어셈블리>
</link>
```

ACTk는 이 사례를 자동으로 처리하고 재생 모드에서 이를 감지하여 경고하고 자동 link.xml 파일 생성(ACTK_WALLHACK_LINK_XML 조건부 컴파일 기호 사용)을 활성화할 수 있는 설정을 엽니다.

- Android에서 [READ_PHONE_STATE](#) 권한 요구 사항을 제거하려면 어떻게 해야 하나요 ?

[ObscuredPrefs / ObscuredFile / ObscuredFilePrefs](#) 에서 장치 잠금 기능을 사용하지 않고 안드로이드 애플리케이션을 구축하는 경우 [ACTk 설정 창](#) 에서 [ACTK_PREVENT_READ_PHONE_STATE](#)를 확인하여 제거하는 것이 좋습니다.

Android의 [READ_PHONE_STATE](#) 권한 요구 사항입니다. 그렇지 않은 경우 ACTk가 장치에 대한 저장 내용을 잠그도록 하려면 이 요구 사항이 필요합니다.

- Android에서 [인터넷](#) 권한 요구 사항을 제거하려면 어떻게 해야 하나요 ?

[TimeCheatingDetector](#)를 사용하지 않는 경우 [ACTk](#) 에서 [ACTK_PREVENT_INTERNET_PERMISSION](#)을 확인하는 것이 좋습니다.

[Android](#)에서 [인터넷](#) 권한 요구 사항을 제거하는 [설정](#) 창입니다. 그렇지 않은 경우 ACTk에서 시간 부정 행위를 감지하려면 이 요구 사항이 필요합니다.

- ACTk를 사용할 때 난독처리가 Unity 빌드를 중단시킵니다.

이름으로 메서드를 호출할 수 있는 Unity의 메서지, Invoke() 및 코루틴을 인식하지 못하는 일부 난독 처리기를 사용하고 있고 난독 처리 장치가 [System.Reflection.ObfuscationAttribute](#) 와 호환되는 경우 속성에 대해 [ACTk 설정 창](#) 에서 [ACTK_EXCLUDE_OBFUSCATION](#)을 확인하여 [\[Obfuscation\(Exclude = true\)\]](#)을 추가하세요.

그러한 방법에 속합니다. 도움이 되지 않으면 저에게 연락해주세요.

- 이전 버전에서 ACTk를 업데이트했는데 이제 ObscuredPrefs를 읽을 수 없습니다.

[ObscuredPrefs](#)를 사용 중이고 ACTk를 업데이트하려는 경우 다음 사항을 고려하십시오. ACTk >= 1.4.0은 ACTk < 1.2.5에서 ObscuredPrefs로 저장된 데이터를 해독할 수 없으므로 버전 < 1.2에서 벗어나지 않도록 하세요. .5를 버전 >= 1.4.0으로 직접 변환합니다. 환경 설정을 새 형식으로 자동 변환하거나 암호 해독 코드를 가져와서 ACT >= 1.4.0에 대한 추가 폴백으로 가져오려면 그 사이의 모든 버전을 사용해야 합니다. 마이그레이션에 문제가 있는 경우 언제든지 저에게 연락해 주시면 원활한 진행을 도와드리겠습니다.

- [Assets/Resources/fndid.bytes](#) 파일 에 대한 병합 충돌이 발생했습니다 .

프로젝트에 버전 제어를 사용하고 있고 [InsertDetector](#)를 사용하는 경우 병합 충돌이 발생할 수 있습니다. 괜찮습니다. VCS의 무시 목록에 해당 파일을 추가하기만 하면 됩니다. [fndid.bytes](#) 는 자동으로 생성되며 시스템마다 다를 수 있습니다.

- 콘솔에 StackOverflowException 오류가 표시됩니다.

다음과 같은 메시지가 표시되는 경우:

StackOverflowException: 요청한 작업으로 인해 스택 오버플로가 발생했습니다.

CodeStage.AntiCheat.ObscuredTypes.ObscuredPrefs.HasKey(System.String 키)(*CodeStage/
AntiCheatToolkit/Scripts/ObscuredTypes/ObscuredPrefs.cs:*에 위치)

클래스뿐만 아니라 ACTk 클래스에서도 실수로 모든 PlayerPrefs.* 호출을 교체했을 가능성이 높습니다. 프로젝트에서 ACTk를 제거하고 다시 가져오면 문제가 해결됩니다.

- [ObscuredString.GetEncrypted\(\)](#) \ [EncryptDecrypt\(\)](#) 메서드 의 결과로 문자열이 잘리거나 너무 짧습니다 .
가려진 문자열에는 암호화된 동안 줄 바꿈, 줄 끝 등과 같은 특수 문자가 포함될 수 있습니다. 따라서 깨진 것처럼 보일 수 있지만 길이와 실제 내용은 여전히 괜찮습니다.
- 1.5.8.0 이하에서 ACTk를 업데이트한 후 손상된 ObscuredFloats/ObscuredDoubles가 보입니다.
이러한 유형은 1.6.0에서 새로운 형식을 얻었으며 저장되거나 직렬화된 인스턴스는 새 형식으로 마이그레이션되어야 합니다.
이를 완료하려면 다음을 사용하십시오.
 - Unity 직렬화된 인스턴스 마이그레이션을 위한 메뉴 명령: **도구 > Code Stage > Anti-Cheat Toolkit > Migrate** • [GetEncrypted\(\)](#)로 저장하기 위한 런타임 API: [ObscuredFloat/ObscuredDouble.MigrateEncrypted\(\)](#)
 - [ACTk 설정](#) 창의 조건부 컴파일 플래그 [ACTK_OBSCURED_AUTO_MIGRATION](#)은 인스턴스가 새 값으로 다시 작성될 때까지 인스턴스에 대한 런타임 확인 및 자동 마이그레이션을 활성화합니다. ObscuredFloat/ObscuredDouble의 성능이 약간 감소합니다.

1.5.1.0 이하에서 업데이트하는 경우 첫 번째 형식 변경 후 1.5.2.0에 추가된 추가 마이그레이션 단계를 수행해야 합니다. 전체 *.*.* - 1.5.2.0 - 1.6.0 마이그레이션을 수행하려면 이전 ACTk 버전을 요청하세요.

- 이전 Android 기기(Android 6 이전)의 [Time Cheating Detector](#) 로그에 [Error response code: 0](#) 또는 [java.io.EOFException](#) 오류가 표시됩니다 .
UnityWebRequest에는 오래된 Android 기기에서만 재현되는 알려진 버그가 있습니다. 요청을 설정해 보세요
이 버그를 해결하려면 HEAD 대신 GET 메서드를 사용하세요.
- adb 로그에 [java.lang.ClassNotFoundException: net.codestage.*](#) 오류가 표시됩니다 .
난독화 또는 축소에서 기본 ACTk Android 플러그인을 제외해야 합니다. 자세한 내용은 [ProGuard 공지](#)를 참조하세요 .
- Android 또는 WebGL의 StreamingAssets에서 ObscuredFile 또는 ObscuredFilePrefs를 읽거나 쓸 수 없습니다. 이것은 이러한 플랫폼의 StreamingAssets 특성으로 인해 예상되는 동작(Unity [매뉴얼](#) 참조) 상서 사항은). [이 문제를 해결하려면 이 간단한 예](#) 와 같이 ObscuredFile을 사용하여 파일을 읽기 전에 StreamingAssets 폴더에서 File.IO 액세스 가능 위치(예: [Application.dataPath](#)) 로 파일을 복사하세요. 아이디어를 보여주지 위해 만들어졌습니다.

마이그레이션 참고사항

Anti-Cheat Toolkit v2021에서 v2023으로 마이그레이션

ACTk v2021에서 v2023으로 업데이트할 때 다음과 같이 더 이상 사용되지 않는 API 대신 새 API를 사용하려면 코드를 업데이트해야 할 수 있습니다.

- 이제 더 이상 사용할 수 있는 [CodeHashGeneratorPostprocessor.Instance](#) 속성이 없습니다.
필요한 모든 속성을 직접:
 - [Instance.callbackOrder](#) => [CodeHashGeneratorPostprocessor.CallbackOrder](#)
 - [Instance.HashesGenerated](#) => [CodeHashGeneratorPostprocessor.HashesGenerated](#)
- [HashGenerated](#) 이벤트 핸들러 서명이 변경되었습니다. [hashedBuilds](#) 인수에 새로운 유형이 있습니다.
 - [BuildHashes\[\] hashedBuilds](#) => [IReadOnlyList<BuildHashes> hashedBuilds](#)
- [BuildHashes.FileHashes](#) 및 [HashGeneratorResult.FileHashes](#) 속성 유형이 Array에서 Array 로 변경되었습니다.
[IReadOnly목록](#):
 - [FileHash\[\] FileHashes](#) => [IReadOnlyList<FileHash> FileHashes](#)

호환성

아래 나열된 몇 가지 예외를 제외한 모든 기능은 알려진 모든 플랫폼에서 완벽하게 작동해야 합니다.

- [InjectionDetector](#)는 Android Mono 및 Standalone Mono 빌드에서만 작동합니다.
- [CodeHashGenerator](#)는 Android 및 Windows 독립 실행형 빌드에서만 작동합니다.
- [AppInstallationSourceValidator](#)는 Android 플랫폼에서만 작동합니다.

플러그인은 독립 실행형 (Win, Mac, Linux, WebGL), iOS, Android, UWP 플랫폼에서 테스트되었습니다.

또한 고객들은 다음 작업을 수행하고 있다고 보고했습니다.

Windows Phone 8, Apple TV(thx [atmuc](#)).

특정 플랫폼에서 플러그인이 작동하지 않는 경우 알려주시면 도와드리겠습니다.

Apple 암호화 수출 규정 호환성

모든 ACTk 기능은 Apple의 수출 규정 준수 및 미국 수출 규정과 호환되며 [ITSAAppUsesNonExemptEncryption](#) plist.info 키를 YES로 설정할 필요가 없습니다 .

이러한 ACTk 기능은 기본적으로 면제("대중 시장") 암호화를 사용합니다.

- ObscuredFile 및 ObscuredFilePrefs
- ObscuredLong, ObscuredULong, ObscuredDouble 및 ObscuredDecimal 유형

"대중 시장" 암호화는 미국 수출 관리 규정(EAR) 섹션 [742.15에 따라 수출 규정에서 면제됩니다](#). 앱에서 언급된 ACTk 기능을 사용하는 경우 Apple App Store 에 게시할 때 앱에서 암호화를 사용하고 있음을 선언해야 하지만 이는 라이선스가 필요하지 않은 면제 암호화입니다.

그러나 [Apple 설명서](#) 에 명시된 [바와 같이 면제 암호화를 사용하려면](#) 미국 정부에 자체 분류를 제출해야 할 수도 있습니다.

앱에서 다른 면제 암호화(예: https / tls / rsa / ssh 등)를 사용하지 않고 ACTk에서 앱이 암호화를 사용한다고 선언해야 하는 경우 조건부 컴파일의 [ACTK_US_EXPORT_COMPATIBLE](#) 옵션을 사용하여 이를 방지할 수 있습니다. 기호 설정.

이는 대칭 암호화를 위해 56비트를 초과하는 키 생성을 방지하므로 더 이상 산업 보안국 상업 통제 목록 카테고리 5 파트 2, 2.a.1.a의 암호화 정의에 속하지 않습니다. Apple App Store에 애플리케이션을 게시할 때 애플리케이션에서 암호화를 사용하고 있음을 선언합니다. , 그래서 ACTk는

하지만 이 방법에는 다음과 같은 부작용이 따른다는 점에 유의하세요.

- ObscuredFile 및 ObscuredFilePrefs 암호화 강도는 128비트 키를 사용하는 AES에서 56비트 키를 사용하는 RC2로 전환하여 약화됩니다(데이터는 여전히 암호화되어 읽을 수 없음).
- 부분 ObscuredLong, ObscuredULong, ObscuredDouble 및 ObscuredDecimal 모호함을 약화시켜 그렇지 않게 만듭니다.
드문 경우지만 완전히 암호화됩니다. [ObscuredCheatingDetector](#)는 아찔든 그들을 감시할 것입니다.

ProGuard 공지

Android 빌드에 대해 축소를 활성화한 경우 제외해야 합니다. 즉, proguard-user.txt에 다음을 추가하세요.

```
-keep 클래스 net.codestage.ack.androidnative.ACTkAndroidRoutines { *; }
-keep 클래스 net.codestage.ack.androidnative.CodeHashGenerator {public void GetCodeHash(...);}
-keep 클래스 net.codestage.ack.androidnative.CodeHashCallback { *; }
```

[도구 > 코드 스테이지 > 치트 방지 툴킷 > proguard-user.txt](#) 구성 메뉴 항목을 사용하여 이 줄을 사용하여 proguard-user.txt 파일을 생성할 수 있습니다 .

타사 라이선스 포함

xxHashSharp

<https://github.com/noricube/xxHashSharp>

BSD 2조 라이선스 (<http://www.opensource.org/licenses/bsd-license.php>)

SharpZipLib

<https://github.com/icsharpcode/SharpZipLib> MIT

라이선스 (<https://opensource.org/licenses/MIT>)

저자의 마지막 말

한 번 더 말씀드립니다. 제 툴킷은 매우 속련되고 동기가 부여된 사기꾼을 막을 수 있는 도구가 아니라는 점을 명심하시기 바랍니다.

실제로 그러한 솔루션은 없습니다. 한 달에 수천 달러의 비용이 드는 거대한 안티 치트 솔루션에도 여전히 결함이 있고 사용자를 속이는 경우가 있습니다.

ACTk는 대부분의 치팅 플레이어에 대해 도움이 될 뿐만 아니라 고급 솔로 치터의 삶을 더 어렵게 만들 것입니다:P

귀하의 요구에 적합한 Anti-Cheat Toolkit을 찾으셔서 귀하의 소중한 시간을 절약해 주시기를 바랍니다!

[리뷰를 남겨주세요](#) 플러그인의 Asset Store 페이지에서 버그 보고서, 기능 제안 및 기타 의견을 포럼이나 아래 지원 연락처를 통해 주저 없이 보내주세요.

시간을 내어 이 문서를 읽어주셔서 감사합니다!

Anti-Cheat 툴킷 링크 및 자원

[에셋스토어](#) | [홈페이지](#) | [API](#) | [법정](#) | [유튜브](#)

지원 연락처: [discord.gg/](https://discord.gg/KrU4psWffA)

KrU4psWffA.codestage.net/

[contacts](#)

support@codestage.net

업데이트 및 뉴스를 구독하세요:



[Discord 공지 채널](#)



[@codestage_net](#)



[@codestage](#)

행운을 빌겠습니다

다. Dmitriy Yukhanov

[Asset Store 게시자](#)

codestage.net

PS #0 Unity 에셋 스토어 활동을 지원하고 매일 행복하게 해준 가족에게 감사 인사를 전하고 싶습니다!

PS #1 [Daniele Giardini](#)에게 큰 감사를 전하고 싶습니다. ([DOTween](#), [HODU](#), [고스커리](#) 그리고 다른 많은 행복을 만들어내는 것들 제작자)에게 이 툴킷에 대한 멋진 로고, 집중적인 도움 및 귀중한 피드백을 제공합니다!